

LECTURE 2
Linked Lists

AGENDA

2.1 Introduction

2.2 Linked Lists

2.3 Singly Linked List (SLL) Implementation in C++

2.4 Main SLL Operations (with Code)

Learning Objectives:



By the end of this lecture, you will be able to:



Define the structure of a Node and a Linked List in C++.

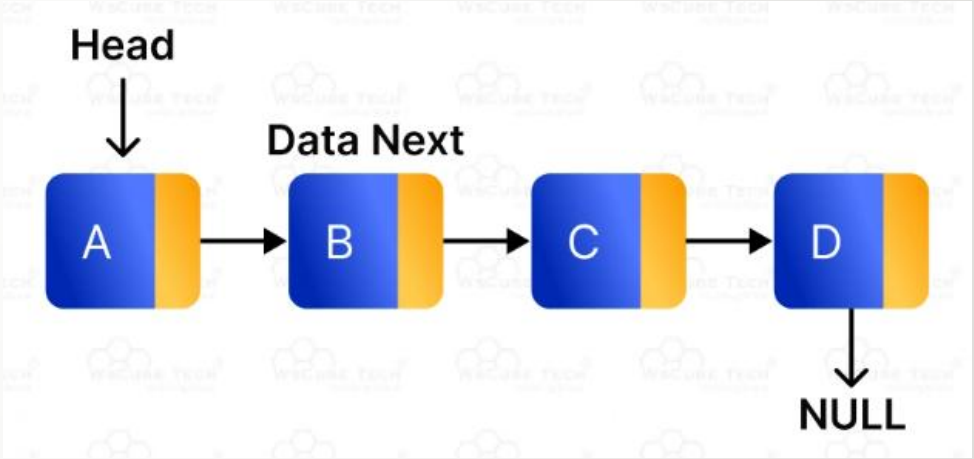


Visualize and implement all basic LL operations (Insert, Delete, Traverse).

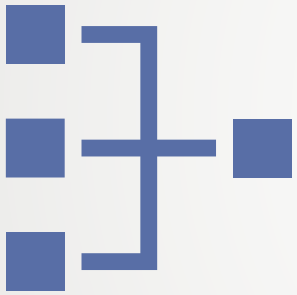
2.1. Introduction: what is a linked list?

Definition: A linear data structure where elements (Nodes) are **not** stored in contiguous memory locations.

Connection: Elements are linked using **Pointers**



Core Components:

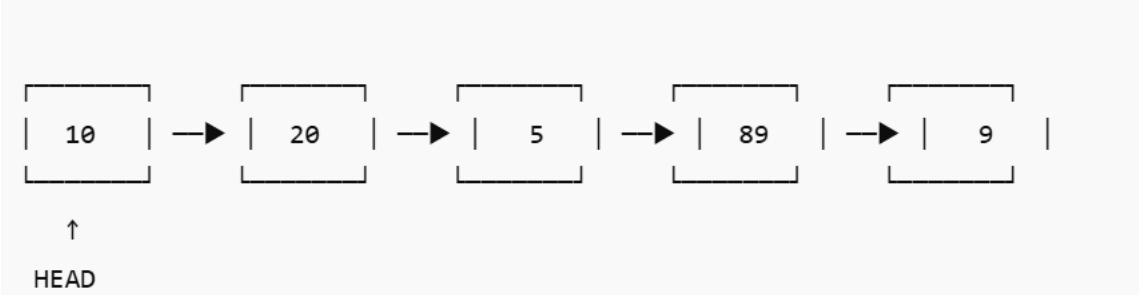


Node: The fundamental element containing data and a link to the next node.

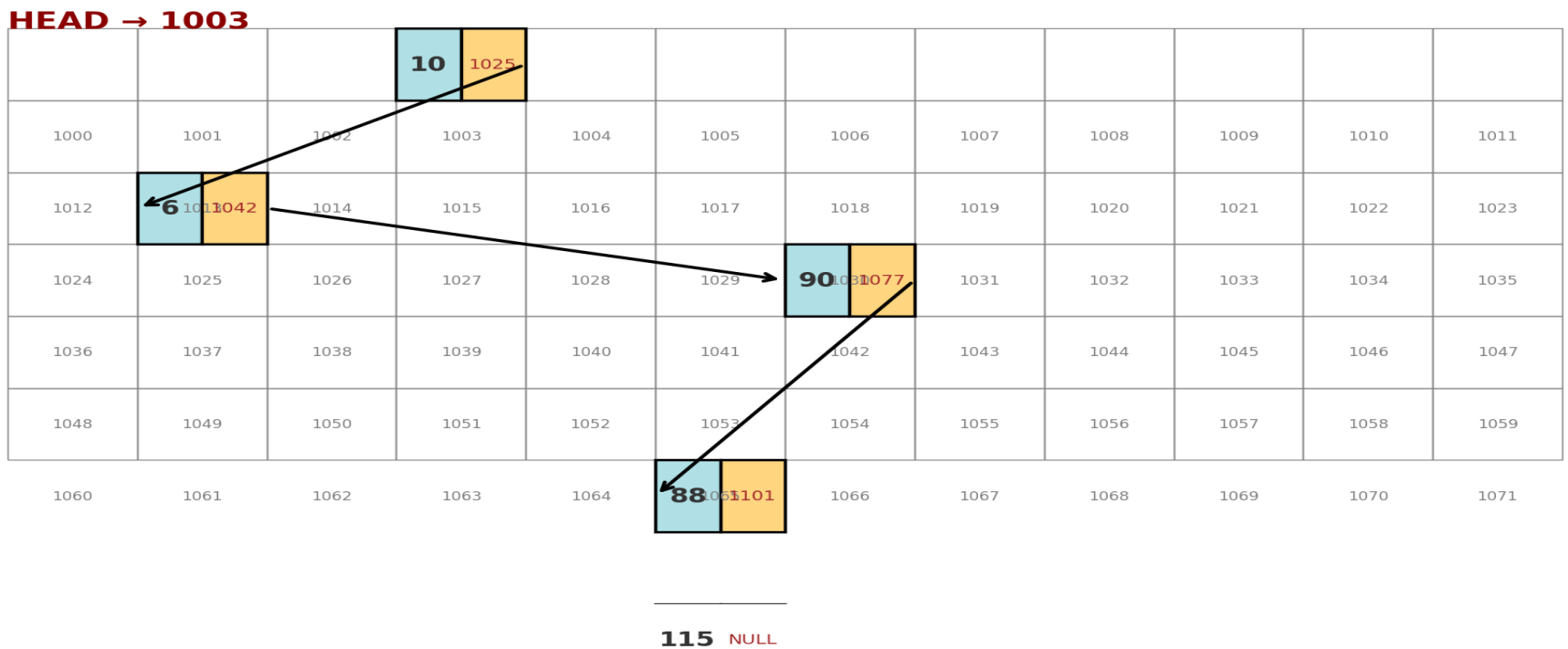


Head: A pointer to the **first Node** in the list. It marks the list's beginning.

```
10 → 20 → 5 → 89 → 9
```



Linked List Memory Representation

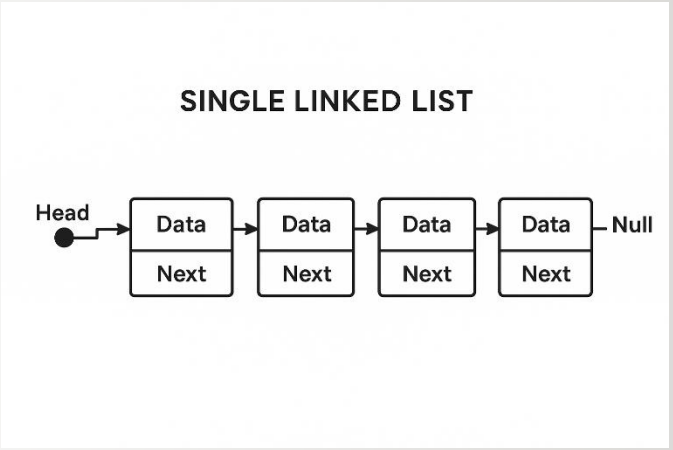


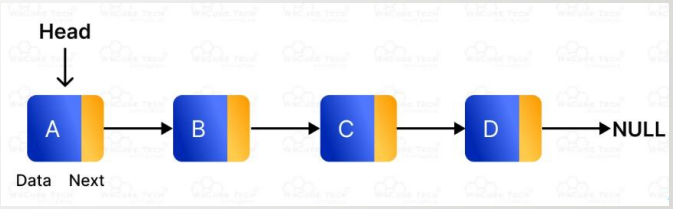
2.2. LINKED LISTS

2.2.1.Singly linked list

A singly linked list is a collection of nodes where each node contains data and a reference to the next node in the sequence. This structure allows for a simple, one-directional traversal from the head to the end of the list.

Operations like adding or removing tasks involve adjusting the 'next' reference of the preceding node, which is straightforward but only allows moving forward through the list.

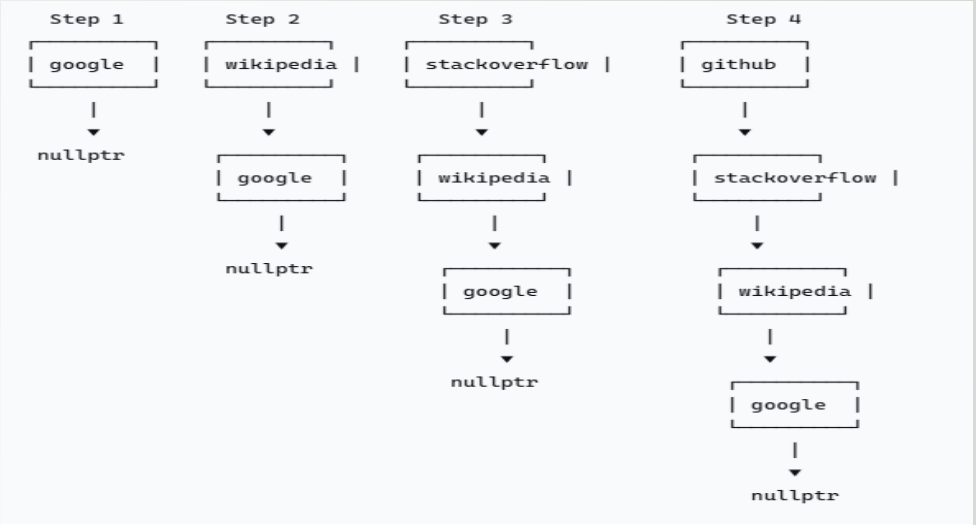


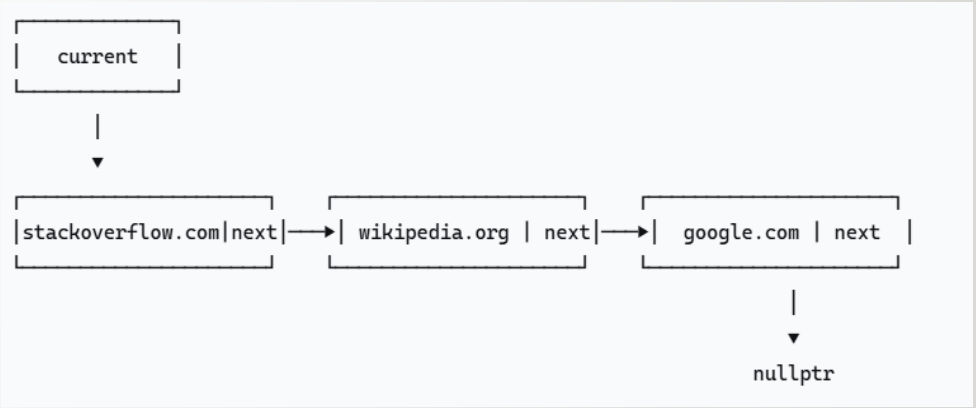


Example of singly linked list

- Web Browser Navigation History

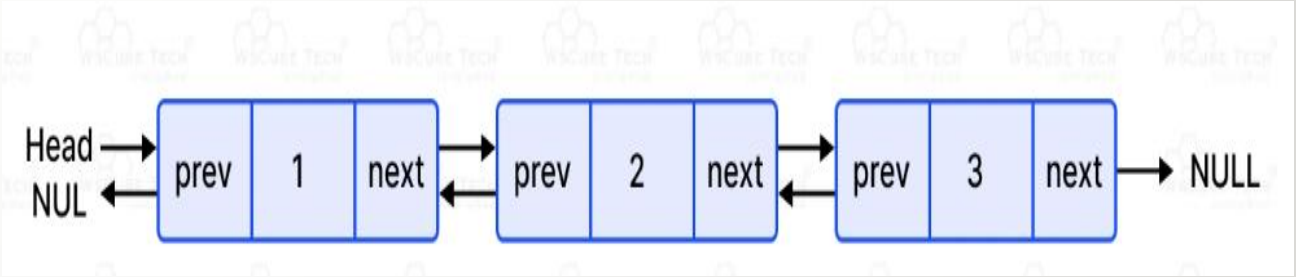
One of the best real-world examples of a singly linked list is the **back/forward navigation** in a web browser.





2.2.2.Doubly Linked List

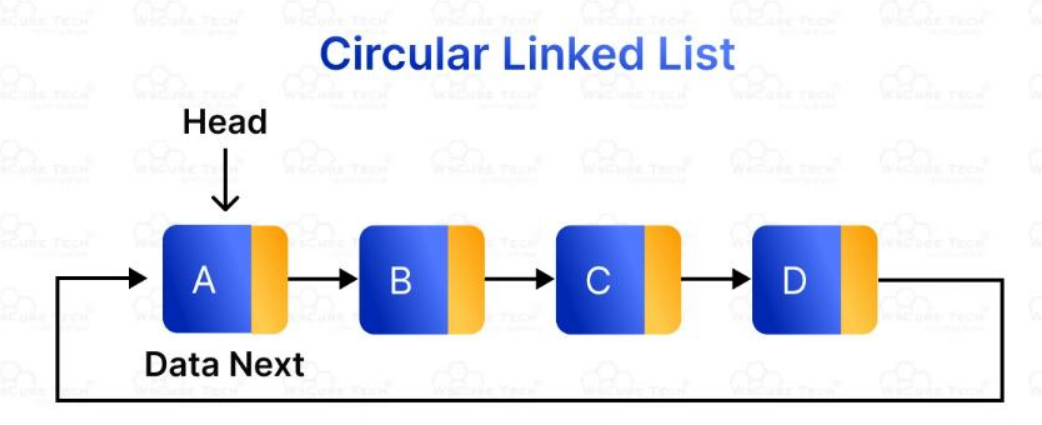
Doubly linked lists expand on singly linked lists by having each node contain two references: one pointing to the next node and one to the previous node. This allows nodes to be traversed in both forward and backward directions.



2.2.3.Circular Linked List

A circular linked list is a type of data structure where each node points to the next node, and the last node points back to the first, forming a circle.

This setup allows for an endless cycle of node traversal, which can be particularly useful in applications where the end of the list naturally reconnects to the beginning.



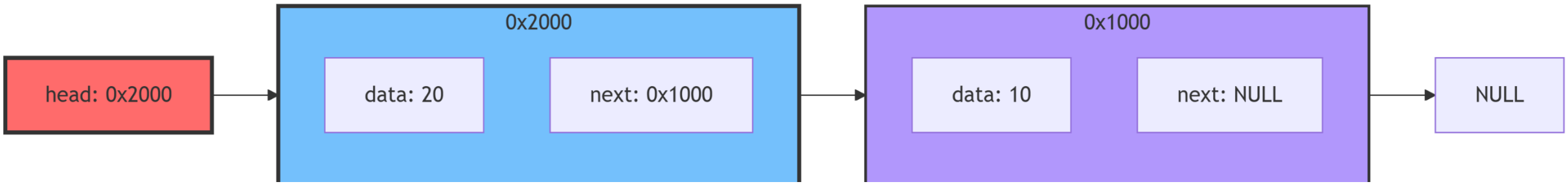
2.3.SINGLE LINKED LIST (SLL)

Single linked list

A single linked list is a linear data structure where each element (node) is connected to the next one via a **pointer**, forming a sequence.

Each node contains two parts:

- **Data**: The value stored in the node.
- **Next**: A pointer to the next node in the list.



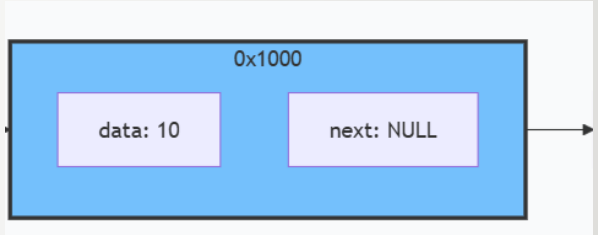
How is the node implemented and represented ?

The node contains two parts:

1. Data part
2. Next pointer

```
struct Node {
    int data;       // Store numeric data
    Node* next;     // Pointer to next node

    // Constructor
    Node(int value) : data(value), next(nullptr) {}
};
```



How to implement the linked list

```
class LinkedList {
private:
    Node* head;       // Pointer to the head of the list
    Node* tail;       // Pointer to the tail of the list

public:
    // Constructor
    LinkedList() : head(nullptr), tail(nullptr) {}
    ……
    ……
    ……
    // Write here all linked list operations (Functions)
    ……
    ……
```

2.4.SLL operations



2.4.SLL operations

A singly linked list is a type of linked list data structure that behaves like a one-way list/chain. The reason it is called a one-way list or one-way chain is that we can only traverse this list in one direction, starting from the head node to the end.

Understanding how to perform operations on linked lists is crucial for using them effectively in programming.

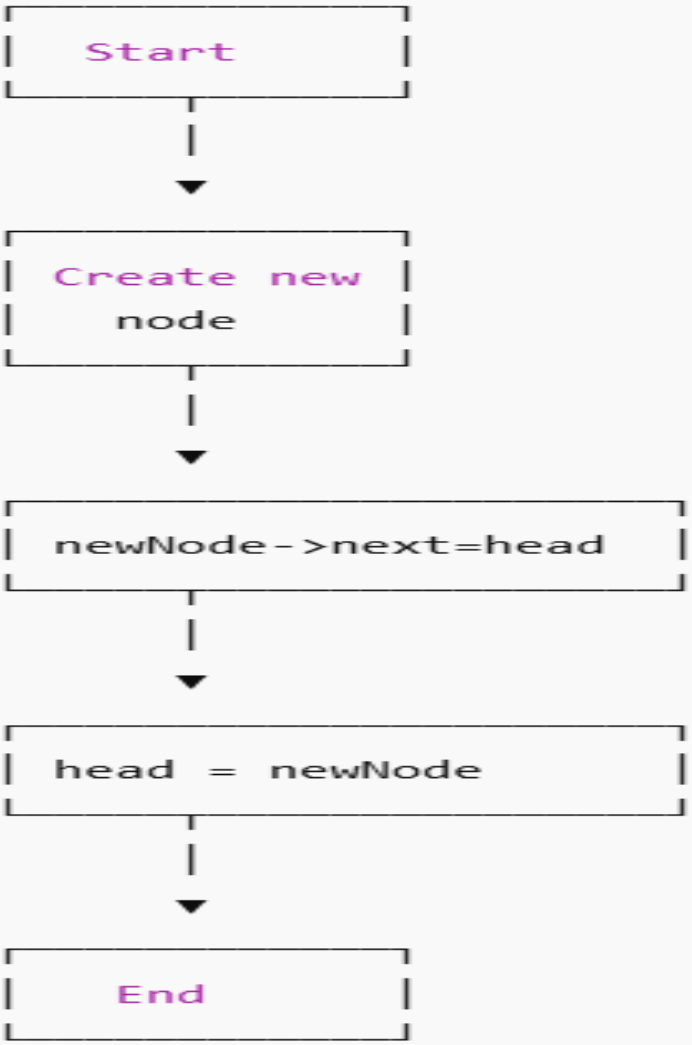
📌 **Main Operations:**

1. **Insertion**
   - At beginning
   - At end
   - At a specific position
2. **Deletion**
   - From beginning
   - From end
   - From a specific position
3. **Traversal (Display all nodes)**
4. **Search (Find a node by value)**

2.4.1.Insertion

2.4.1.1. Insertion (1. Insert at the beginning)

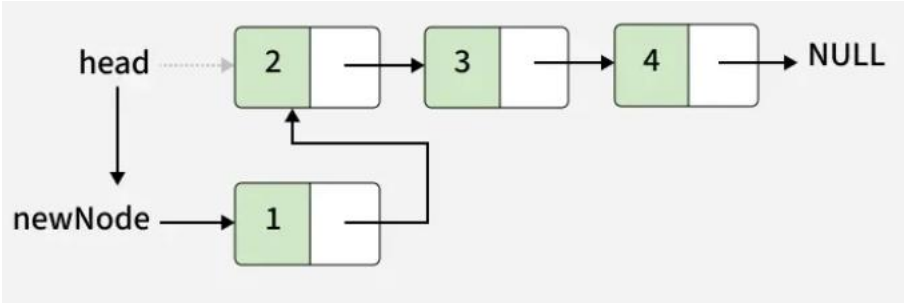
- Create a new node.
- Point its next to the current head.
- Update head to this new node.



2.4.1.1. Insertion (1. Insert at the beginning) cont..

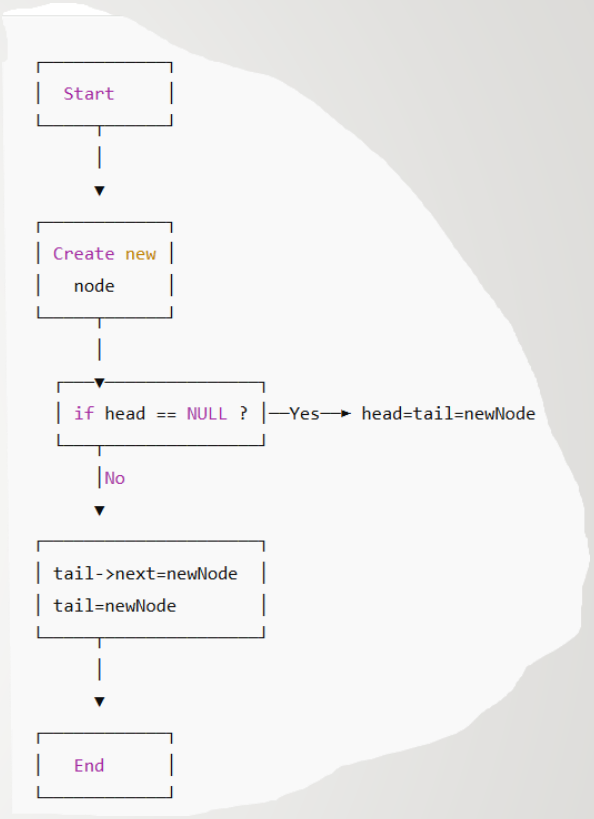
```
void insertAtBeginning(int value) {
    // Create a new node with the given value
    Node* newNode = new Node(value);

    // Check if list is empty by checking if head is null
    if (head == nullptr) {
        // If list is empty, new node becomes both head and tail
        head = tail = newNode;
    } else {
        // If list is not empty:
        // 1. New node points to current head
        // 2. Update head to point to new node
        newNode->next = head;
        head = newNode;
    }
    cout << "Inserted " << value << " at beginning" << endl;
}
```



2.4.1.2.Insertion (2. Insert at the end)

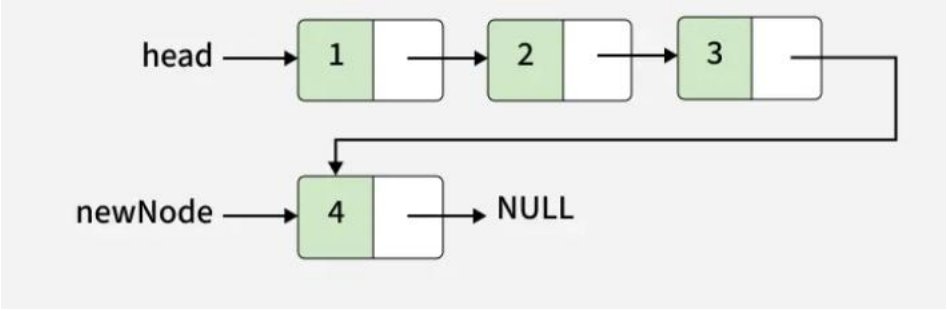
- Traverse to the last node.
- Point its next to the new node.
- Set new node’s next = NULL.



Insertion (2. Insert at the end) CONT..

```
void insertAtEnd(int value) {
    // Create a new node with the given value
    Node* newNode = new Node(value);

    // Check if list is empty by checking if tail is null
    if (tail == nullptr) {
        // If list is empty, new node becomes both head and tail
        head = tail = newNode;
    } else {
        // If list is not empty:
        // 1. Current tail points to new node
        // 2. Update tail to point to new node
        tail->next = newNode;
        tail = newNode;
    }
    cout << "Inserted " << value << " at end" << endl;
}
```

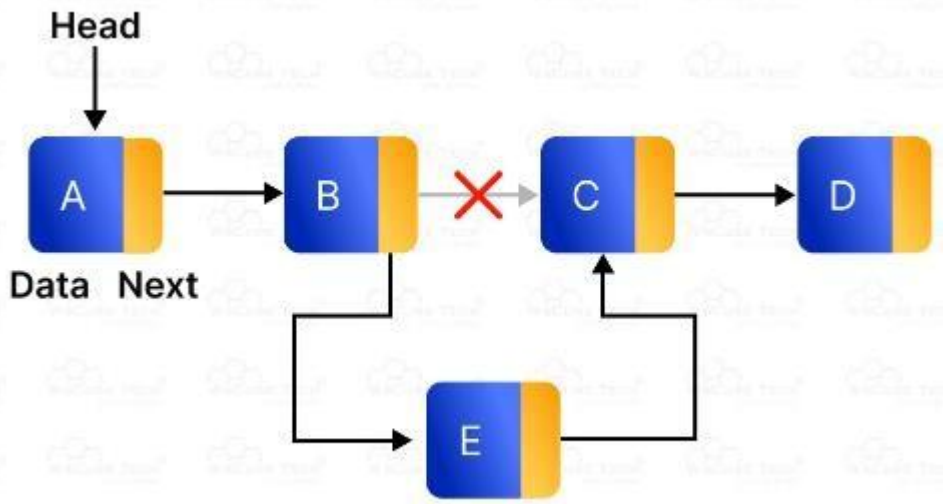


2.4.1.3. Insertion (3. Insert at a position)

- Create a new node.
- Traverse to the node before the desired position.
- Set the new node’s next pointer to the current node’s next.
- Update the current node’s next pointer to the new node.

2.4.1.3. Insertion (3. Insert at a position) CONT..

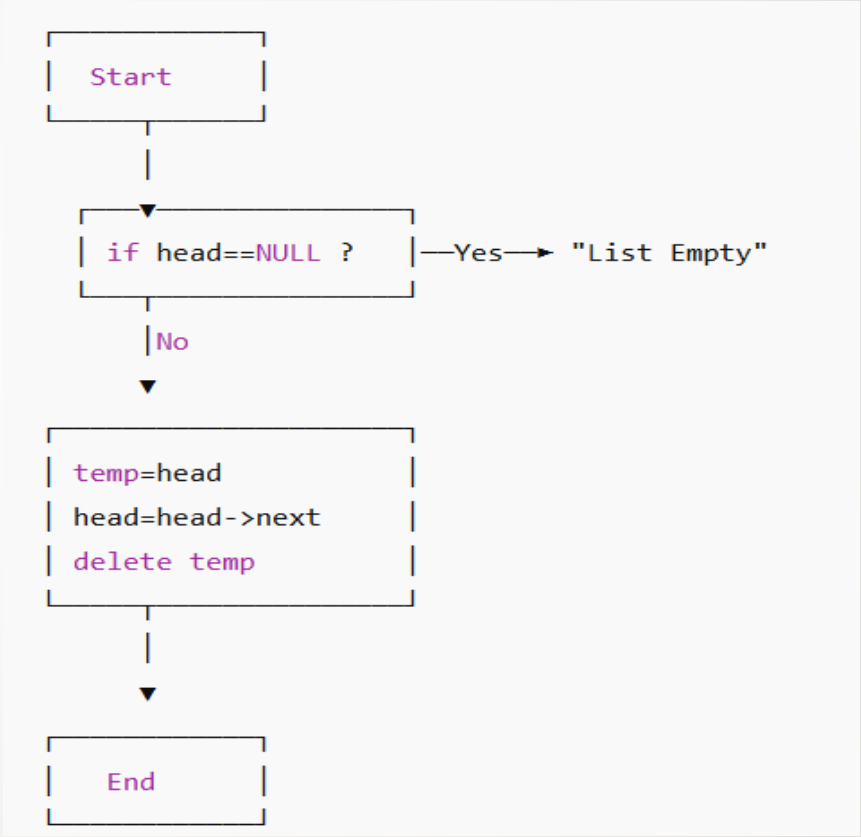
```
// Function to insert a node at a specific position
   void insertAtflosition(int value, int position) {
        Node* newNode = new Node(value);      // Create a new node
        if (position == 1) {
            // Insert at the beginning if position is 1
            newNode->next = head;
            head = newNode; }
        else {
            Node* temp = head;
            // Traverse to the position before the insertion point
            for (int i = 1; i < position - 1 fifi temp != nullptr; i++) {
                temp = temp->next;
            }
            if (temp == nullptr) {
                std::cout << "position out of bounds" << std::endl; }
            else {
                newNode->next = temp->next; // Link the new node to the next node
                temp->next = newNode; // Link the previous node to the new node
            }
        }
```



2.4.2.Deletion

2.4.2.1. Deletion (1. delete from the beginning)

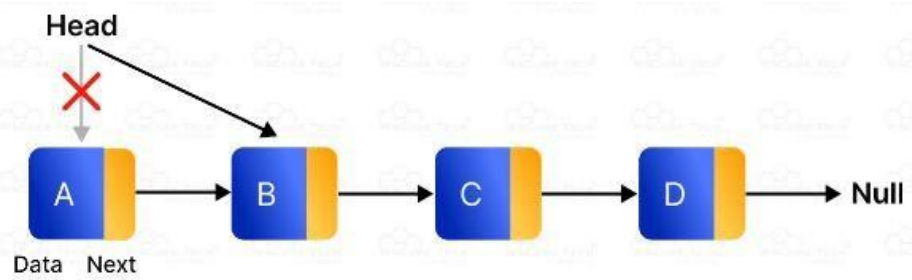
- Move head to the next node.



Deletion (1. delete from the beginning) cont..

```
void deleteFromBeginning() {
    // Check if list is empty
    if (head == nullptr) {
        cout << "List is empty! Nothing to delete.\n";
        return;
    }

    // Store the node to be deleted and update head
    Node* temp = head;
    head = head->next;
    cout << "Deleted " << temp->data << " from the beginning.\n";
    delete temp; // Free memory
}
```



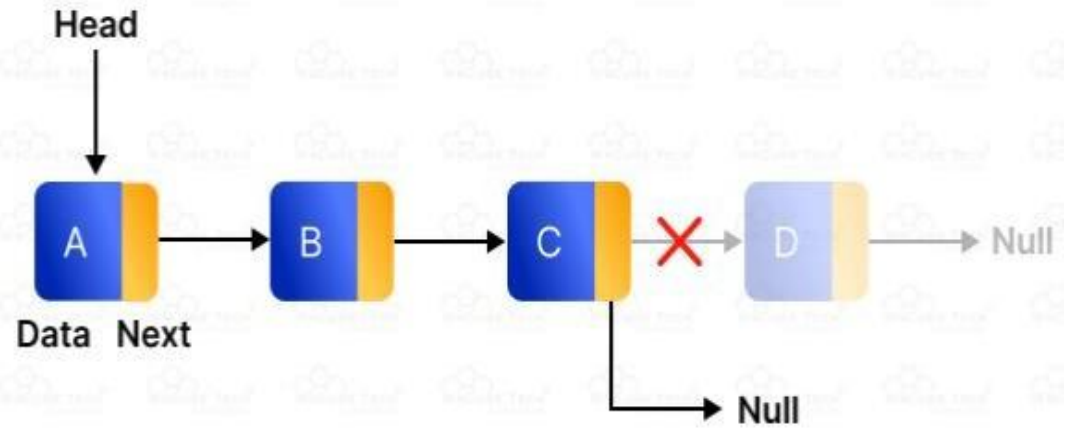
2.4.2.2. Deletion (2. Delete from the end)

1. **Check if the list is empty**
   - If `head == NULL`, the list is empty → nothing to delete.
2. **Check if the list has only one node**
   - If `head->next == NULL`, then:
     - Delete `head`
     - Set `head = NULL`
3. **Traverse to the second-last node**
   - Start from `head`
   - Keep moving until `temp->next->next == NULL`

     (meaning `temp` is pointing to the node before the last).
4. **Delete the last node**
   - Save pointer to the last node: `toDelete = temp->next`
   - Set `temp->next = NULL` (break the link)
   - Free `toDelete`

Deletion (2. Delete from the end) cont..

```
void deleteAtEnd() {
    if (head == nullptr) {
        cout << "List is empty, nothing to delete." << endl;
        return;
    }
    if (head->next == nullptr) {
        delete head;   // If there's only one node
        head = nullptr;
        return;
    }
    Node* temp = head;
    // Traverse to the second last node
    while (temp->next->next != nullptr) {
        temp = temp->next;
    }
    delete temp->next;   // Delete the last node
    temp->next = nullptr;    // Set the new last node's next to nullptr
    }
```

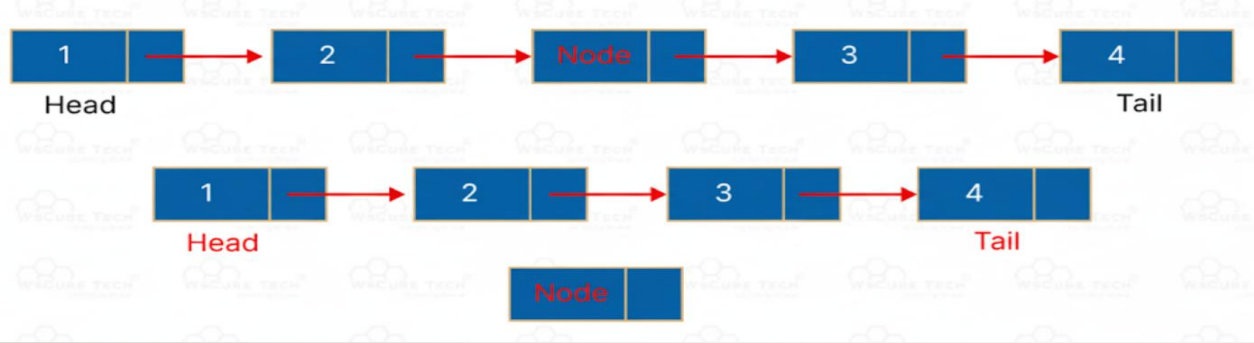


2.4.2.3. Deletion (3. Delete from specific location)

- Check if the list is empty.
- Traverse to the node before the desired position.
- Update the node's next pointer to skip the node at the desired position.

1. **Check if the list is empty**
   - If `head == NULL` → no deletion possible.
2. **If deleting the first node (position = 1)**
   - Store `head` in `toDelete`
   - Move `head = head->next`
   - Free `toDelete`
3. **Otherwise, traverse to the node just before the given position**
   - Start from `head`
   - Move `(pos - 2)` times (because you want the previous node).
   - If during traversal you reach `NULL` → position is invalid.
4. **Delete the node at the given position**
   - Save pointer: `toDelete = prev->next`
   - Update link: `prev->next = toDelete->next`
   - Free `toDelete`

2.4.2.3. DELETION (3. DELETE FROM SPECIFIC LOCATION) CONT..




```
void deleteAtPosition(Node* &head, int pos) {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1) {
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
        return;
    }

    Node* temp = head;
    for (int i = 1; temp != NULL && i < pos - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        cout << "Invalid position\n";
        return;
    }

    Node* toDelete = temp->next;
    temp->next = temp->next->next;
    delete toDelete;
}
```

2.4.3.Traversal

2.4.3.Traversal (displaying all nodes)

Traversal is the process of systematically visiting every single node in the linked list exactly once, starting from the beginning (the **head**) and moving towards the end (the **tail**).

Steps :

- Initialize pointer to head
- Check if list is empty
- Visit current node
- Move to next node
- Repeat until pointer becomes NULL
- End traversal

```
void traverse(Node* head) {
    Node* temp = head;          // Step 1: start from head
    while (temp != NULL) {      // Step 2: loop until NULL
        cout << temp->data << " "; // Step 3: visit current node
        temp = temp->next;         // Step 4: move to next node
    }
    cout << endl;
}
```

2.4.4.Search

2.4.4. Search (find any node by value)

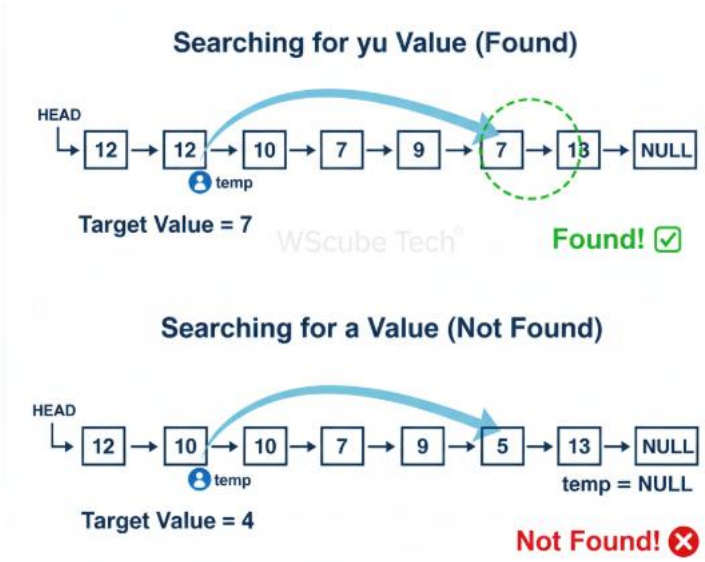
- Searching in a linked list means **finding whether a given key (value) exists in the list or not**.
  It can be:
- **Successful search** → if the value is found.
- **Unsuccessful search** → if the value is not in the list.

Steps :

1. Initialize a pointer to the head node.
2. Check if the list is empty. If yes → stop (element not found).
3. Compare the current node's data with the key.
4. If match is found → return success (element found).
5. If not, move pointer to the next node.
6. Repeat steps 3–5 until the pointer becomes NULL.
7. If end is reached and no match → return failure (element not found).

2.4.4. Search (find any node by value) cont..

```
void search(Node* head, int key) {
    Node* temp = head;   // Start from head
    while (temp != NULL) {
        if (temp->data == key) {
            cout << key << " found in the list." << endl;
            return;  // Exit after finding
        }
        temp = temp->next;
    }
    cout << key << " not found in the list." << endl;
}
```



Thanks

